

# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

## ASSESSMENT TOOL 2 – Pseudocode Writing Task

|                         |                                                                                                                                                                       |                      |                                             |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|---------------------------------------------|
| <b>Course Code:</b>     | CSA06                                                                                                                                                                 | <b>Course Title:</b> | Design and Analysis of Algorithms           |
| <b>CO Assessed:</b>     | CO5 – Naturalization (BL5, BL6)                                                                                                                                       | <b>Assessment:</b>   | Assessment Tool 2 – Pseudocode Writing Task |
| <b>Weightage:</b>       | 30% of CIA                                                                                                                                                            | <b>Total Marks:</b>  | 100                                         |
| <b>CO5 Statement:</b>   | <i>Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.</i> |                      |                                             |
| <b>Student Name:</b>    | Naveen Kumar S                                                                                                                                                        | <b>Reg. No.:</b>     | 192421030                                   |
| <b>Section / Batch:</b> | B Tech IT                                                                                                                                                             | <b>Date:</b>         | 27.08.2026                                  |

### Instructions to Students

- Answer the following question with proper pseudocode and logical explanation.
- Clearly specify the algorithmic approach and complexity class wherever applicable.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) for complexity analysis.
- Include state-space representation, recursion steps, and pruning conditions in backtracking problems.
- Neat presentation and step-by-step justification are mandatory.

| Question Type                  | Focus Area                                                                                                        | Questions |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------|-----------|
| <b>Pseudocode Writing Task</b> | Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete. | Q1        |

### SECTION A – Pseudocode Writing Task

#### Code of Conduct

*I Naveen Kumar S - 192421030(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Naveen Kumar S

### RUBRICS FOR CALCULATION

| Criteria | Weightage | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|----------|-----------|---------------------|----------------------|----------------------|---------------------|
|----------|-----------|---------------------|----------------------|----------------------|---------------------|

|                           |     |                                                                       |                                           |                              |                                 |
|---------------------------|-----|-----------------------------------------------------------------------|-------------------------------------------|------------------------------|---------------------------------|
| Problem Understanding     | 20% | Identifies all inputs, outputs, constraints accurately                | Minor missing elements                    | Partial understanding        | Incorrect interpretation        |
| Algorithm Design          | 30% | Complete, correct, and logically sound solution                       | Minor logical gaps                        | Partially correct logic      | Incorrect approach              |
| Use of Control Structures | 20% | Efficient and appropriate use of loops, conditions, functions/modules | Minor inefficiencies                      | Limited or basic usage       | Incorrect or missing constructs |
| Pseudocode Structure      | 10% | Well-structured, clear, consistent format, easy to follow             | Mostly clear with minor issues            | Difficult to follow in parts | Poorly structured and unclear   |
| Completeness              | 20% | Handles all cases; optimized approach                                 | Handles most cases; reasonable efficiency | Handles basic cases only     | Incomplete or inefficient       |

### Marks Evaluation:

|                           |     |             |                      |     |  |
|---------------------------|-----|-------------|----------------------|-----|--|
|                           |     |             | Pseudocode Structure | 10% |  |
|                           |     |             | Completeness         | 20% |  |
| Criteria                  | Wt. | Lev<br>Exer | Total Marks (100):   |     |  |
| Problem Understanding     | 20% |             |                      |     |  |
| Algorithm Design          | 30% |             |                      |     |  |
| Use of Control Structures | 20% |             |                      |     |  |

|                   |                    |                    |
|-------------------|--------------------|--------------------|
| Faculty In-charge | Course Coordinator | Head of Department |
| Name:             | Name:              | Name:              |
| Signature: _____  | Signature: _____   | Signature: _____   |
| Date: _____       | Date: _____        | Date: _____        |

**SIMATS**  
ENGINEERING

**SIMATS Online Programming Lab**  
DAA PROGRAMMING

**NAVEEN KUMAR S**  
192421030RC

```

2  result = []
3  nums.sort() # Sort to make pruning effective and avoid duplicat
4
5  def backtrack(start, current_subset, current_sum):
6      # Base case: found a valid subset
7      if current_sum == target:
8          result.append(list(current_subset))
9          return
10
11     # Pruning: if sum exceeds target, stop this branch
12     if current_sum > target:
13         return
14
15     for i in range(start, len(nums)):
16         # Skip duplicate elements at the same recursion level
17         if i > start and nums[i] == nums[i - 1]:
18             continue
19
20         # Pruning before including the element
21         if current_sum + nums[i] > target:
22             continue
23
24         # Include nums[i]
25         current_subset.append(nums[i])
26         backtrack(i + 1, current_subset, current_sum + nums[i])
27         # Exclude nums[i] (backtrack)

```

6 7 8 9 2  
11

**Output**

Enter number of elements: Enter  
 5 integers separated by spaces:  
 Enter target sum:  
 All valid subsets with sum = 11  
 :  
 [2, 9]

## EXPLANATION :

The **Subset Sum Problem** asks us to find **all subsets of a given set of integers whose elements add up exactly to a specified target sum.**

- **Input:** A set/list of integers and a target sum.
- **Output:** All subsets whose sum is equal to the target.
- **Backtracking:** The algorithm considers each element in two ways: **include it** in the current subset or **exclude it**.
- **Recursion:** These include/exclude choices are repeated recursively until all possible subsets are explored.
- **Pruning:** If the current sum becomes greater than the target, that branch can be stopped early **when all numbers are non-negative**.
- **No duplication:** Each element is considered once at each level, so every subset is generated systematically without repeating the same selection.
- **Completeness:** Backtracking explores both inclusion and exclusion choices, ensuring that **all valid subsets** are found.
- **Complexity:** In the worst case, there are  $2n2^n$  possible subsets, so the time complexity is

approximately  $O(2^n)$ .

**Example:**

For the set  $\{2, 3, 5, 7\}$  and target 10, valid subsets include  $\{3, 7\}$  and  $\{2, 3, 5\}$  because both have a sum of 10.

**In short:** Backtracking efficiently searches through all possible subsets by **including/excluding each element and pruning impossible branches**, while ensuring every valid subset is found.